Decisões Técnicas — Projeto Agente de Pesquisa

Documento de referência · Gerado em 01/05/2026 · Versão 1.0

> Este documento consolida as decisões tomadas sobre stack de desenvolvimento, orçamento, ferramentas de produtividade e controle de horas para o projeto **Agente de Pesquisa** (FastAPI + WhatsApp/Instagram + Evolution API + OpenRouter).

1. Contexto do Projeto

Item	Detalhe
Repositório	github.com/geterborges100-tech/agente-pesquisa
Stack principal	Python / FastAPI / PostgreSQL / Docker / Evolution API / OpenRouter (Gemini)
Ambiente de dev	Windows 11 + WSL2 + Docker Desktop
Ambiente de produção	Oracle Cloud — Ubuntu 22.04 — 1 GB RAM
Webhook local	ngrok (a ser reduzido com mocks)
Branch ativa	fix/consent-persistence-e-gemini
Perfil	Solo developer
Orçamento total	US$ 40/mês (desenvolvimento + IA + runtime)

2. Decisão de Orçamento

> **Orçamento total definido: US$ 40/mês**
> Esse valor cobre ferramentas de desenvolvimento, assistente de IA e reserva para runtime do produto.

Destino	Ferramenta	Custo	Papel
Editor / IDE	Cursor Pro	US$ 20/mês	Edição multi-arquivo, contexto de codebase, agente no editor
Chat / Raciocínio	ChatLLM Teams Basic	US$ 10/mês	Arquitetura, debugging, revisão, planejamento
Edição via terminal	Aider	Grátis	Alterações guiadas no repositório via CLI

Tarefas simples	DeepSeek (gratuito)	Grátis	Rascunhos, documentação, perguntas genéricas
Runtime / testes	OpenRouter / reserva	US$ 10/ mês	LLM do agente em testes e produção

> **Atenção:** Não contratar SaaS pago de controle de horas neste momento. O orçamento restante (US$ 10) deve ser preservado para o runtime do produto.

3. Stack de Desenvolvimento — Decisão Final

3.1 Ferramentas pagas

Ferramenta	Função	Por que escolhida
Cursor Pro	IDE principal com IA integrada	Maior velocidade absoluta de edição, entende o codebase inteiro, multi-arquivo, agente no editor
ChatLLM Teams Basic	Assistente de raciocínio externo	Revisão de arquitetura, debugging difícil, análise de logs, planejamento de sprint

3.2 Ferramentas gratuitas

Ferramenta	Função	Observação
Aider	Edição de repositório via terminal	Complementa o Cursor para alterações guiadas por CLI
DeepSeek (gratuito)	Tarefas leves	Rascunhos, documentação, refactors triviais, perguntas genéricas
pytest	Testes unitários	Já presente no projeto
ruff	Linting e formatação	Rápido, substitui flake8 + black
pre-commit	Validação antes do commit	Garante qualidade mínima automaticamente
GitHub Actions	CI/CD	Lint + testes + deploy automático

3.3 Quando usar cada ferramenta

Situação	Ferramenta recomendada
Editar código no dia a dia	Cursor Pro
Refatorar múltiplos arquivos	Cursor Pro + Aider
Revisar arquitetura ou decisão técnica	ChatLLM Teams Basic

Depurar erro difícil / analisar log	ChatLLM Teams Basic
Rascunho de função simples	DeepSeek gratuito
Documentação / texto genérico	DeepSeek gratuito
Código com dados sensíveis / LGPD	Cursor Pro ou ChatLLM (nunca modelo gratuito)

---

4. Controle de Horas — Decisão Final

Ferramenta escolhida: TimeTracker (open source)
Repositório: github.com/DRYTRIX/TimeTracker
Motivo: melhor suporte a previsão/forecast, burn rate, orçamento por projeto e comparação previsto vs realizado.

4.1 Por que TimeTracker e não outras opções

Ferramenta	Previsão/Forecast	Custo	Decisão
TimeTracker	Forte — burn rate, alertas, confiança	Grátis (self-hosted)	**ESCOLHIDA**
Kimai	Moderada — orçamentos básicos	Grátis (self-hosted)	Descartada
OpenProject	Boa, mas pesada	Grátis (self-hosted)	Descartada — pesada demais
Clockify Pro	Boa	US$ 7,99/usuário/mês	Descartada — custo extra
Harvest	Boa	US$ 12/usuário/mês	Descartada — custo alto

4.2 Funcionalidades do TimeTracker relevantes para o projeto

- Timers de um clique e entradas manuais
- Metas semanais de horas
- Orçamento por projeto com alertas
- Forecast de data de conclusão baseado em burn rate
- Nível de confiança da previsão (alta / média / baixa)
- Comparação planejado vs realizado
- Suporte a PostgreSQL em produção
- Deploy via Docker Compose

4.3 Próximos passos com TimeTracker

- Clonar o repositório e subir via Docker Compose localmente

- Configurar PostgreSQL como banco de dados
- Criar projetos espelhando as sprints do Agente de Pesquisa
- Registrar estimativas antes de iniciar cada tarefa
- Registrar horas realizadas ao finalizar
- Avaliar em 7 dias se atende ou se vale construir módulo próprio

---

5. Melhorias Prioritárias de Infraestrutura

> Antes de investir mais em ferramentas, estas melhorias têm impacto direto na produtividade e estabilidade do projeto.

Prioridade	Melhoria	Impacto
1	Padronizar versão do Python em dev e produção	Elimina bugs de ambiente
2	Rodar tudo dentro do Docker localmente	Paridade com produção
3	Mockar webhook da Evolution API	Reduz dependência do ngrok
4	Reduzir chamadas desnecessárias ao LLM	Reduz custo de runtime
5	Automatizar deploy seguro para Oracle	Elimina deploy manual
6	Atualizar README com estrutura real do projeto	Reduz custo cognitivo

---

6. Orientação de Commit — TimeTracker

Siga o padrão **Conventional Commits** já adotado no projeto. Abaixo o fluxo recomendado para adicionar o TimeTracker ao repositório.

6.1 Estrutura de pastas sugerida

```
agente-pesquisa/
├── app/
├── docs/
├── tests/
├── timetracker/ <-- novo: submódulo ou docker-compose separado
│ ├── docker-compose.yml
│ └── .env.example
├── docker-compose.yml <-- principal (agente)
└── README.md
```

6.2 Sequência de commits recomendada

```
# 1. Criar branch dedicada
git checkout -b feat/timetracker-setup
```

```
# 2. Adicionar TimeTracker como submódulo Git
git submodule add https://github.com/DRYTRIX/TimeTracker timetracker
git commit -m "chore: add TimeTracker as git submodule for time tracking"
```

```
# 3. Criar arquivo de configuração local
cp timetracker/.env.example timetracker/.env
# editar .env com credenciais do PostgreSQL local
git add timetracker/.env.example
git commit -m "chore(timetracker): add env example for local setup"
```

```
# 4. Documentar decisão no projeto
# criar docs/decisoes/timetracker.md com contexto da escolha
git add docs/decisoes/timetracker.md
git commit -m "docs: add TimeTracker decision record and setup guide"
```

```
# 5. Abrir PR para main
git push origin feat/timetracker-setup
# abrir PR no GitHub com título:
feat: integrate TimeTracker for time forecasting and tracking
```

6.3 Padrão de mensagens de commit para o projeto

Prefixo	Quando usar	Exemplo
feat:	Nova funcionalidade	feat: add consent gate to webhook flow
fix:	Correção de bug	fix: resolve Gemini timeout on long prompts
chore:	Configuração, dependências, infra	chore: add TimeTracker as git submodule
docs:	Documentação	docs: update README with real project structure
refactor:	Refatoração sem mudança de comportamento	refactor: extract prompt builder to separate module
test:	Testes	test: add unit tests for ai_engine consent check
ci:	GitHub Actions / CI/CD	ci: add lint and test workflow on push

7. Resumo das Decisões

Tema	Decisão
Orçamento total	US$ 40/mês
IDE principal	Cursor Pro — US$ 20/mês
Chat / raciocínio	ChatLLM Teams Basic — US$ 10/mês
Edição via terminal	Aider — grátis
Tarefas leves	DeepSeek gratuito
Reserva runtime	US$ 10/mês para OpenRouter / testes
Controle de horas	TimeTracker (open source, self-hosted)
Banco de dados do TimeTracker	PostgreSQL (mesmo ecossistema do projeto)
Padrão de commits	Conventional Commits
CI/CD	GitHub Actions
Próxima prioridade técnica	Padronizar Python + mockar webhook + automatizar deploy